

QUANTUM COMPUTING

Hands-On Lab Guide

14 Real Experiments You Can Run on Your Computer Today

No PhD Required. No Cloud Account Needed. Just Python and Curiosity.

Platform	Origin Quantum QPanda3 (pyqpanda3)
Setup Time	5 minutes with Perplexity Pro
Prerequisites	Python 3.10+, curiosity
GitHub	github.com/zkrishna12/Perplexity-Max-Quantum-Computing-Setup

Table of Contents

1	What Is This Guide?	3
2	What Will You Get Out of This?	3
3	Quick Start Setup (5 Minutes)	4
4	How to Use with Perplexity Pro	5
5	Experiment Guide	6
	5.1 Quantum Coin Flip	6
	5.2 Quantum Entanglement	7
	5.3 Quantum Random Number Generator	7
	5.4 Grover's Search Algorithm	8
	5.5 Quantum Teleportation	8
	5.6 Advanced Algorithms (DJ, QFT, VQE, Noise)	9
	5.7 Quantum Coin Game	9
	5.8 Quantum Password Cracker	10
	5.9 Quantum Error Correction	10
	5.10 Quantum Random Walk	11
	5.11 BB84 Quantum Key Distribution	11
	5.12 Superdense Coding	12
6	What Is Quantum Computing? (Plain English)	13
7	About Origin Quantum and QPanda3	15
8	Free Learning Resources	16
9	Where Quantum Is Heading	17

1. What Is This Guide?

This is a **hands-on quantum computing lab** that you can run entirely on your own computer. Instead of reading theory for months, you run 14 real quantum experiments and see quantum mechanics in action through charts, numbers, and results you can touch.

Every experiment was built and tested using **Perplexity Pro** (an AI assistant) in under 20 minutes. The AI handled all the coding, debugging, and chart generation. You just learn.

TIP: You do NOT need to understand the code to learn from these experiments. Run them, look at the charts, read the printed explanations. The understanding comes from seeing the results, not from reading the source code.

Who Is This For?

You Are	What You Will Get
A curious beginner	See quantum computing work without needing a physics degree
An IT professional	Hands-on skills to discuss quantum confidently with clients
A student	Portfolio-ready experiments and a foundation for advanced courses
A developer	Working quantum code you can modify, extend, and build upon

2. What Will You Get Out of This?

After running all 14 experiments, you will understand these quantum concepts not just in theory, but because you **saw them work**:

- **Superposition** — A qubit can be 0 AND 1 at the same time — you will see a coin land exactly 50/50
- **Entanglement** — Two qubits linked so measuring one instantly determines the other — you will see perfectly correlated results
- **Quantum Search** — Finding a needle in a haystack quadratically faster — you will see Grover's algorithm crack passwords
- **Quantum Cryptography** — Unbreakable encryption using physics, not math — you will catch an eavesdropper red-handed
- **Error Correction** — Protecting fragile qubits from noise — you will break a qubit and watch it heal itself

-
- **Quantum Speedup** — Why quantum walks explore faster — you will see the dramatic difference in spread

CAREER VALUE: Quantum computing skills are among the fastest-growing in tech. McKinsey estimates the quantum computing market will reach \$850 billion by 2040. Understanding these fundamentals now puts you years ahead of your peers.

3. Quick Start Setup (5 Minutes)

Prerequisites

- **Python 3.10 or higher** — Download from python.org if you do not have it
- **pip** — Comes with Python automatically
- **A terminal** — Command Prompt (Windows), Terminal (Mac/Linux)

Step 1: Clone the Repository

```
git clone https://github.com/zkrishna12/Perplexity-Max-Quantum-Computing-Setup.git
cd Perplexity-Max-Quantum-Computing-Setup
```

Step 2: Create a Virtual Environment

```
python -m venv quantum-env

# Windows:
quantum-env\Scripts\activate

# Mac / Linux:
source quantum-env/bin/activate
```

Step 3: Install Dependencies

```
pip install pyqpanda3 matplotlib numpy
```

This installs Origin Quantum's QPanda3 simulator, plus matplotlib for charts and numpy for math. Total download: about 50 MB.

Step 4: Run Your First Experiment

```
python experiments/01_coin_flip.py
```

You will see printed results in your terminal and a chart saved in the **charts/** folder. That is it. You just ran your first quantum experiment.

IMPORTANT: Everything runs locally on your computer. No cloud account, no registration, no API key needed. QPanda3 simulates a quantum computer right on your machine.

4. How to Use with Perplexity Pro

This entire lab was built using **Perplexity Pro** as an AI coding assistant. Here is how you can do the same — and even create new experiments beyond what is in this guide.

Getting Started

1. Open Perplexity Pro

Go to perplexity.ai and start a new conversation.

2. Tell it what you want

Try: *"I have Python installed. Help me set up Origin Quantum QPanda3 and run a quantum coin flip experiment. Explain everything in plain English."*

3. Let it work

Perplexity will create the virtual environment, install dependencies, write code, run it, and generate charts — all automatically.

4. Ask questions

Say things like *"What does this result mean?"* or *"Run the password cracker experiment next"*. It will explain everything as it goes.

Example Prompts You Can Try

- "Run experiment #8 (password cracker) and explain what it means"
- "Show me how quantum entanglement works with a live demo"
- "Can quantum computing break my WiFi password?"
- "Create a new experiment that shows quantum tunneling"
- "Explain the BB84 results like I am 12 years old"

TIME SAVED: What would take weeks of reading documentation and debugging took less than 20 minutes with Perplexity Pro. The AI handles the technical complexity so you can focus on understanding the concepts.

5. Experiment Guide

Below is every experiment in this lab. For each one, you will find: what it does, how to run it, what to look for in the results, and what it means in the real world.

5.1 Quantum Coin Flip

File: experiments/01_coin_flip.py

Run: `python experiments/01_coin_flip.py`

What it does: Flips a quantum coin 1,000 times using a Hadamard gate. The coin lands approximately 50% heads and 50% tails — but unlike a classical coin, the randomness is guaranteed by the laws of physics, not by how hard you flip.

What to look for: A bar chart showing roughly 500 heads and 500 tails. The split will never be exactly 50/50 (that is probability), but it will be close.

Real-world connection: Quantum random number generators are already used in cryptography, lotteries, and scientific simulations where true randomness matters.

5.2 Quantum Entanglement

File: experiments/02_entanglement.py

Run: `python experiments/02_entanglement.py`

What it does: Creates a Bell state — two qubits that are perfectly linked. When you measure one, the other instantly shows the matching result, no matter how far apart they are.

What to look for: Results show ONLY '00' and '11' — never '01' or '10'. The qubits are perfectly correlated.

Real-world connection: Entanglement is the backbone of quantum internet, quantum teleportation, and superdense coding. China's Micius satellite already uses it for long-distance quantum communication.

5.3 Quantum Random Number Generator

File: experiments/03_random_numbers.py

Run: `python experiments/03_random_numbers.py`

What it does: Uses 8 qubits in superposition to generate truly random numbers between 0 and 255. Unlike your computer's pseudo-random numbers (which follow a hidden pattern), these are fundamentally unpredictable.

What to look for: A histogram showing 1,000 random numbers spread fairly uniformly across 0-255. No clustering, no pattern.

Real-world connection: Companies like ID Quantique sell quantum random number generators for banking, gaming, and government security applications.

5.4 Grover's Search Algorithm

File: experiments/04_grovers_search.py

Run: `python experiments/04_grovers_search.py`

What it does: Searches through 4 possible answers and finds the correct one in just 1 quantum step. A classical computer would need up to 4 tries.

What to look for: The target answer ('11') appears with nearly 100% probability. All other options are close to 0%.

Real-world connection: For a database of 1 billion items, Grover's algorithm reduces search from 1 billion steps to about 25,000 — a 40,000x speedup.

5.5 Quantum Teleportation

File: experiments/05_teleportation.py

Run: `python experiments/05_teleportation.py`

What it does: Transfers the state of one qubit to another using entanglement and classical communication. The original qubit is destroyed in the process (no cloning allowed in quantum mechanics).

What to look for: Multiple scenarios showing the teleported qubit matches the original state every time.

Real-world connection: Quantum teleportation is essential for building quantum networks and a future quantum internet. It has been demonstrated over 1,400 km using China's Micius satellite.

5.6 Advanced Algorithms

File: experiments/06_advanced_algorithms.py

Run: `python experiments/06_advanced_algorithms.py`

What it does: Four experiments in one: (1) Deutsch-Jozsa algorithm determines a function type in 1 query, (2) Quantum Fourier Transform does frequency analysis on qubits, (3) VQE finds the lowest energy of a simple system, (4) Noise simulation shows how errors affect real quantum hardware.

What to look for: Four subplot charts. DJ shows 100% correct identification. VQE shows energy converging to minimum. Noise chart shows fidelity dropping with increased error rate.

Real-world connection: VQE is used today by pharmaceutical companies to simulate drug molecules. QFT is the basis of Shor's algorithm that could break RSA encryption.

5.7 Quantum Coin Game

File: experiments/07_coin_game.py

Run: `python experiments/07_coin_game.py`

What it does: A game where a quantum player competes against a classical player. The quantum player uses superposition to create a strategy that wins 100% of the time, while the classical player can only win 50%.

What to look for: Classical player wins about 50% of games. Quantum player wins exactly 100%. Every single time.

Real-world connection: Demonstrates quantum advantage in game theory and decision-making. Similar principles apply to quantum strategies in auctions and negotiations.

5.8 Quantum Password Cracker

File: experiments/08_password_cracker.py

Run: `python experiments/08_password_cracker.py`

What it does: Uses Grover's algorithm to find secret 2-bit passwords. Tests all 4 possible passwords and finds each one with 100% accuracy in a single quantum step.

What to look for: Four charts, each with one green bar (the found password) at nearly 1,000 out of 1,000 runs. Plus a speed comparison chart showing quantum speedup.

Real-world connection: For 1 billion passwords: classical needs 1 billion tries, quantum needs about 25,000. That is 40,000x faster. This is why governments are already upgrading to quantum-safe encryption.

5.9 Quantum Error Correction

File: experiments/09_error_correction.py

Run: `python experiments/09_error_correction.py`

What it does: Stores one qubit's data across three qubits. Then deliberately corrupts each qubit one at a time. The error correction code detects and fixes every error automatically — 100% recovery.

What to look for: Bar charts showing 100% recovery rate for both $|0\rangle$ and $|1\rangle$ states, regardless of which qubit was corrupted.

Real-world connection: This is THE biggest challenge in quantum computing. Google's Willow chip uses error correction to make its 105 qubits reliable. We need ~1 million physical qubits to get ~1,000

reliable logical qubits.

5.10 Quantum Random Walk

File: experiments/10_random_walk.py

Run: `python experiments/10_random_walk.py`

What it does: Compares how a classical particle and a quantum particle spread out over time. Classical: bell curve centered near the start. Quantum: two peaks near the edges, spreading much faster.

What to look for: Three charts (10, 20, 50 steps) showing the classical bell curve vs the quantum two-peaked distribution. The quantum particle covers far more ground.

Real-world connection: This faster exploration is why quantum algorithms can search faster. Grover's algorithm is essentially a quantum walk through a database.

5.11 BB84 Key Distribution

File: experiments/11_bb84_key_distribution.py

Run: `python experiments/11_bb84_key_distribution.py`

What it does: Alice and Bob create a secret encryption key using quantum physics. Without an eavesdropper: 0% errors. With an eavesdropper (Eve): about 25% errors — Eve is caught immediately.

What to look for: Side-by-side comparison: clean key exchange (green) vs corrupted exchange with Eve (red bars showing errors). Error rate chart showing 0% vs ~25%.

Real-world connection: BB84 is the world's first quantum cryptography protocol (1984). Already used by banks in Switzerland, government networks in China, and being deployed across Europe and India.

5.12 Superdense Coding

File: experiments/12_superdense_coding.py

Run: `python experiments/12_superdense_coding.py`

What it does: Alice sends 2 classical bits of information by manipulating just 1 qubit. She can send all four messages (00, 01, 10, 11), and Bob decodes each one perfectly.

What to look for: Four charts showing 100% accuracy for each message. Every green bar reaches 1,000/1,000.

Real-world connection: Could theoretically double the capacity of communication channels. It is the reverse of quantum teleportation — teleportation sends 1 qubit using 2 classical bits, superdense

coding sends 2 classical bits using 1 qubit.

5.5 Our Story: How This Was Built in 20 Minutes

This entire lab — 14 experiments, charts, this PDF, and the GitHub repository — was built in a single conversation with **Perplexity Pro**. Here is exactly what happened, step by step, so you can see the power of AI-assisted learning.

The Actual Workflow

Step	What We Said	What Perplexity Did	Time
1	"Help me set up quantum computing on my machine"	Evaluated 3 platforms (Qiskit, Cirq, QPanda3), recommended QPanda3, created virtual environment, installed all dependencies	3 min
2	"What can I test?"	Created 5 beginner experiments (coin flip, entanglement, random numbers, search, teleportation), ran them all live, generated charts	5 min
3	"What does this mean?"	Explained every result in plain English with real-world analogies (no jargon)	2 min
4	"Give me more experiments"	Created a menu of 20 experiments. Then built and ran 9 more advanced ones: password cracker, error correction, BB84, superdense coding, quantum walks, coin game, and more	8 min
5	"Package this for GitHub with a PDF guide"	Created clean experiment files, README.md, requirements.txt, and this comprehensive PDF guide with resources	5 min

What Would This Take Without AI?

Task	Traditional Way	With Perplexity Pro
Research quantum platforms	2-3 hours reading docs	2 minutes (evaluated 3 platforms automatically)
Install and configure	30-60 min (troubleshooting errors)	1 minute (handled all dependencies)
Write 14 experiments	2-3 days (learning API, debugging)	10 minutes (wrote, tested, fixed bugs live)
Generate charts	3-4 hours (matplotlib learning curve)	Automatic (included in each experiment)

Create documentation	1-2 days	5 minutes (PDF + README generated)
Total	3-5 days	~20 minutes

KEY INSIGHT: The AI agent is not just a chatbot. It actually runs code on your machine, debugs errors in real time, generates charts, and iterates until everything works. When the superdense coding chart had a bug, Perplexity Pro detected the error, diagnosed it (a tuple/dict type mismatch), fixed the code, and re-ran it — all automatically. That is the difference between an AI assistant and an AI agent.

How to Reproduce This Yourself

You can recreate this entire experience or extend it further:

- **Start a new conversation** on perplexity.ai with Pro enabled
- **Say:** *"Set up quantum computing on my machine with QPanda3 and run experiments"*
- **Ask for explanations:** *"Explain this in plain English"* or *"Give me real-world examples"*
- **Go beyond this lab:** *"Create a quantum experiment that simulates a drug molecule"*
- **Debug instantly:** If anything breaks, just paste the error — the agent will fix it live

6. What Is Quantum Computing? (Plain English)

A regular computer thinks in **bits** — each bit is either a 0 or a 1. Like a light switch: on or off. A quantum computer thinks in **qubits** — each qubit can be 0, 1, or **both at the same time**. This is called **superposition**.

Imagine looking for a specific book in a library with 1 million books.

- **Classical computer:** Checks one book at a time. Could take up to 1 million checks.
- **Quantum computer:** Checks all books simultaneously. Finds the right one in about 1,000 checks.

The 5 Key Concepts

Superposition

A qubit exists in multiple states at once until you measure it. Like a coin spinning in the air — it is neither heads nor tails until it lands. This lets quantum computers try many solutions simultaneously.

Entanglement

Two qubits can be linked so that measuring one instantly determines the other, regardless of distance. Einstein called this "spooky action at a distance." It is the secret sauce behind quantum communication and many quantum algorithms.

Interference

Quantum states can reinforce or cancel each other out, like waves in water. Quantum algorithms use this to amplify correct answers and cancel wrong ones — that is how Grover's search works.

Measurement

When you measure a qubit, the superposition collapses to a definite 0 or 1. This is irreversible. The art of quantum computing is setting things up so the answer you want has the highest probability when you measure.

No Cloning

You cannot make a perfect copy of an unknown quantum state. This seems like a limitation, but it is what makes quantum cryptography possible — an eavesdropper cannot copy your key without disturbing it.

What Quantum Computers Are Good At

Domain	Application
Optimization	Finding the best route, schedule, or portfolio out of billions of possibilities

Simulation	Modeling molecules for drug discovery and materials science
Cryptography	Breaking current encryption (Shor's) and creating unbreakable encryption (BB84)
Machine Learning	Speeding up certain AI training and pattern recognition tasks
Search	Finding items in unsorted databases quadratically faster (Grover's)

What Quantum Computers Are NOT Good At

Quantum computers will NOT replace your laptop. They are not faster at everyday tasks like browsing the web, editing documents, or running apps. They are specialized tools for specific types of problems where classical computers are fundamentally slow.

7. About Origin Quantum and QPanda3

This lab uses **QPanda3** (Quantum Programming Architecture for NISQ Device Application v3), developed by **Origin Quantum** — China's leading quantum computing company founded in 2017. QPanda3 is open-source and freely available.

Why QPanda3?

Feature	Detail
Speed	20x faster circuit construction than Qiskit (benchmarked in peer-reviewed research) ¹
Installation	One command: <code>pip install pyqpanda3</code>
Hardware	Connected to Origin Wukong, a 72-qubit superconducting quantum processor
Local Simulation	Full quantum simulation on your computer — no cloud account needed
Platform Support	Windows, Mac, and Linux
Language	Python — the most popular programming language in the world

1. QPanda3 benchmarks from: <https://arxiv.org/html/2504.02455v1>

8. Free Learning Resources

Want to go deeper? Here are the best free resources, organized by type.

Online Courses (Free)

Course	Platform	Level
Quantum Computing for Everyone ²	edX (UChicago)	Beginner
Qiskit Textbook ³	IBM (free online)	Beginner-Adv
Quantum 101 ⁴	Microsoft Learn	Beginner
QC Fundamentals ⁵	Linux Foundation	Beginner
Quantum Mechanics & Computation ⁶	edX (UC Berkeley)	Intermediate
Quantum Machine Learning ⁷	Coursera (U of Toronto)	Intermediate

YouTube Channels

- **Quantum Soar** — Simple algorithm explanations, great for beginners
- **Qiskit (IBM)** — Official IBM quantum channel with coding tutorials
- **Quantum Sense** — Strong on mathematical foundations
- **freeCodeCamp** — Full 1.5-hour beginner quantum computing course⁸
- **Google Quantum AI** — Latest breakthroughs and hardware demos
- **MIT OpenCourseWare** — University-level quantum mechanics lectures

Books (Free Online)

- **Quantum Computing for the Quantum Curious** — Open access, needs only high school physics⁹
- **Introduction to Classical and Quantum Computing (Tom Wong)** — Only needs trigonometry as prerequisite
- **Learn Quantum Computation using Qiskit** — Free IBM textbook with runnable code examples³
- **Quantum Computing: Lecture Notes (Ronald de Wolf)** — Theoretical computer science perspective

Tools and Platforms

- **Origin Quantum QPanda3** — What this lab uses — pip install pyqpanda3¹⁰
- **IBM Quantum Experience** — Free access to real quantum computers¹¹

-
- **Google Cirq** — Google's open-source quantum framework
 - **Microsoft QDK** — Q# language and Azure Quantum tools⁴
 - **Quantum Computing Playground** — Browser-based GPU-accelerated simulator

2. edX UChicago: <https://www.edx.org/learn/quantum-computing/the-university-of-chicago-quantum-computing-for-everyone>

3. IBM Qiskit Textbook: <https://github.com/Qiskit/textbook>

4. Microsoft Learn Quantum: <https://learn.microsoft.com/en-us/azure/quantum/>

5. Linux Foundation: <https://training.linuxfoundation.org/training/fundamentals-of-quantum-computing/>

6. UC Berkeley Quantum (edX): <https://www.edx.org/learn/quantum-mechanics>

7. Coursera QML: <https://www.coursera.org/learn/quantum-machine-learning>

8. freeCodeCamp Quantum Course: <https://www.youtube.com/watch?v=tsbCSkvHhMo>

9. Quantum Computing for the Quantum Curious: <https://archive.org/details/oopen-20.500.12657-48236>

10. Origin Quantum QPanda3: <https://originqc.com/developer-tools/qpanda>

11. IBM Quantum: <https://quantum.ibm.com>

9. Where Quantum Computing Is Heading

The Timeline

When	What	Impact
Now (2024-2026)	NISQ era: 100-1000 noisy qubits	Research, proof-of-concept, early optimization
Near (2027-2030)	Early error correction, 1000+ qubits	Drug discovery, materials science, finance
Medium (2030-2035)	Fault-tolerant quantum computing	Breaking current encryption, quantum AI
Long (2035+)	Large-scale quantum advantage	Revolution in science, medicine, energy

How It Will Touch Your Life

- **Already happening:** Your bank is upgrading to quantum-safe encryption. Drug companies used quantum simulations during COVID vaccine research.
- **Next 3-5 years:** Better route planning in navigation apps. Longer phone battery life (quantum-designed materials). Smarter fraud detection on credit cards.
- **5-15 years:** Cheaper fertilizers (quantum-designed). Dramatically better weather forecasting. Personalized medicine designed for your specific body chemistry.

Your Next Steps

- **Run all 14 experiments** — You already have everything installed. Go through them one by one.
- **Modify an experiment** — Change a parameter, add a qubit, try a different oracle. Break things. That is how you learn.
- **Take a free course** — Start with the freeCodeCamp YouTube course (1.5 hours) or the Qiskit Textbook.
- **Join a community** — Reddit's r/QuantumComputing, Qiskit Slack, or IBM Quantum Network.
- **Build something new** — Use Perplexity Pro to help you create your own quantum experiments.

Happy experimenting. You now have a working quantum lab on your computer. The best way to understand quantum computing is to play with it — and that is exactly what this guide is for.

GitHub: github.com/zkrishna12/Perplexity-Max-Quantum-Computing-Setup

Built with [Perplexity Pro](#) | Powered by [Origin Quantum QPanda3](#)